

Laboratório 3 - Dados Relacionais e Joins

Pedro José

```
library(tidyverse)
```

Warning: pacote 'tidyverse' foi compilado no R versão 4.5.3

```
— Attaching core tidyverse packages — tidyverse
2.0.0 —
```

```
✓ dplyr      1.1.4    ✓ readr      2.1.5
✓ forcats    1.0.0    ✓ stringr    1.5.1
✓ ggplot2    4.0.3    ✓ tibble     3.2.1
✓ lubridate  1.9.4    ✓ tidyr      1.3.1
✓ purrr      1.0.4
```

```
— Conflicts —
```

```
tidyverse_conflicts() —
```

```
✗ dplyr::filter() masks stats::filter()
```

```
✗ dplyr::lag() masks stats::lag()
```

```
ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(leaflet)
```

Warning: pacote 'leaflet' foi compilado no R versão 4.5.3

```
#importando os 3 arquivos
```

```
flights <- read_csv("flights.csv")
```

```
Rows: 5819079 Columns: 31
```

```
— Column specification
```

```
Delimiter: ","
```

```
chr (11): AIRLINE, TAIL_NUMBER, ORIGIN_AIRPORT, DESTINATION_AIRPORT, SCHEDULED...
```

```
dbl (20): YEAR, MONTH, DAY, DAY_OF_WEEK, FLIGHT_NUMBER, DEPARTURE_DELAY, TAX...
```

```
ℹ Use `spec()` to retrieve the full column specification for this data.
```

```
ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
airlines <- read_csv("airlines.csv")
```

```
Rows: 14 Columns: 2
```

```
— Column specification
```

```
Delimiter: ","
```

```
chr (2): IATA_CODE, AIRLINE
```

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
airports <- read_csv("airports.csv")
```

```
Rows: 322 Columns: 7  
— Column specification
```

```
Delimiter: ","  
chr (5): IATA_CODE, AIRPORT, CITY, STATE, COUNTRY  
dbl (2): LATITUDE, LONGITUDE
```

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
#selecionando as colunas desejadas  
airports <- airports %>%  
  select(IATA_CODE, CITY, STATE, LATITUDE, LONGITUDE)
```

```
flights <- flights %>%  
  select(DESTINATION_AIRPORT, ARRIVAL_DELAY)
```

```
glimpse(airports)
```

```
Rows: 322  
Columns: 5  
$ IATA_CODE <chr> "ABE", "ABI", "ABQ", "ABR", "ABY", "ACK", "ACT", "ACV",  
"ACY...  
$ CITY      <chr> "Allentown", "Abilene", "Albuquerque", "Aberdeen",  
"Albany",...  
$ STATE     <chr> "PA", "TX", "NM", "SD", "GA", "MA", "TX", "CA", "NJ",  
"AK", ...  
$ LATITUDE  <dbl> 40.65236, 32.41132, 35.04022, 45.44906, 31.53552,  
41.25305, ...  
$ LONGITUDE <dbl> -75.44040, -99.68190, -106.60919, -98.42183, -84.19447,  
-70....
```

```
glimpse(flights)
```

```
Rows: 5,819,079  
Columns: 2  
$ DESTINATION_AIRPORT <chr> "SEA", "PBI", "CLT", "MIA", "ANC", "MSP",  
"MSP", "...  
$ ARRIVAL_DELAY      <dbl> -22, -9, 5, -9, -21, 8, -17, -10, -13, -15, -  
30, -...
```

```
# Definir o tamanho do chunk (1 milhão de linhas)  
chunk_size <- 1000000
```

```

# Ver quantas linhas tem o flights
total_linhas <- nrow(flights)

# Ver o total de linhas
total_linhas

[1] 5819079

#Resolvendo a c,d,e,f
library(readr)

#definindo a função que processa cada chunk,
processar_chunk <- function(chunk, pos) {
  chunk %>%
    select(DESTINATION_AIRPORT, ARRIVAL_DELAY) %>%
    filter(!str_starts(DESTINATION_AIRPORT, "1")) %>%
    drop_na() %>%
    group_by(DESTINATION_AIRPORT) %>%
    summarise(
      soma_atraso = sum(ARRIVAL_DELAY, na.rm = TRUE),
      n = n()
    )
}

#lendo e processando em chunks de 1 milhão cada

resultados <- read_csv_chunked(
  "flights.csv",
  callback = DataFrameCallback$new(processar_chunk),
  chunk_size = 1000000
)

```

— Column specification

```

cols(
  .default = col_double(),
  AIRLINE = col_character(),
  TAIL_NUMBER = col_character(),
  ORIGIN_AIRPORT = col_character(),
  DESTINATION_AIRPORT = col_character(),
  SCHEDULED_DEPARTURE = col_character(),
  DEPARTURE_TIME = col_character(),
  WHEELS_OFF = col_character(),
  WHEELS_ON = col_character(),
  SCHEDULED_ARRIVAL = col_character(),
  ARRIVAL_TIME = col_character(),
  CANCELLATION_REASON = col_character()
)

```

```

)
i Use `spec()` for the full column specifications.

#calculando a média final por aeroporto
media_atraso <- resultados %>%
  group_by(DESTINATION_AIRPORT) %>%
  summarise(
    soma_total = sum(soma_atraso),
    n_total = sum(n),
    media_atraso = soma_total / n_total
  )

#incluindo city e state na tabela flights
# coluna chave de flights = Destination_airport
#coluna chave de airports = IATA_Code

flights_sem_airport <- anti_join(flights, airports, by =
c("DESTINATION_AIRPORT" = "IATA_CODE"))

flights <- flights %>%
  left_join(airports, by = c("DESTINATION_AIRPORT" = "IATA_CODE"))

glimpse(flights)

Rows: 5,819,079
Columns: 6
$ DESTINATION_AIRPORT <chr> "SEA", "PBI", "CLT", "MIA", "ANC", "MSP",
"MSP", "...
$ ARRIVAL_DELAY <dbl> -22, -9, 5, -9, -21, 8, -17, -10, -13, -15, -
30, -...
$ CITY <chr> "Seattle", "West Palm Beach", "Charlotte",
"Miami"...
$ STATE <chr> "WA", "FL", "NC", "FL", "AK", "MN", "MN",
"NC", "T...
$ LATITUDE <dbl> 47.44898, 26.68316, 35.21401, 25.79325,
61.17432, ...
$ LONGITUDE <dbl> -122.30931, -80.09559, -80.94313, -80.29056,
-149....

#Quantos aeroportos tem por estado?
aero_por_estado <- airports %>%
  group_by(STATE) %>%
  summarise(num_aero = n()) %>% #esse n() equivale ao nrow(), mas
dentro do summarise usamos n()
  arrange(num_aero)

head(aero_por_estado)

# A tibble: 6 × 2
  STATE num_aero

```

```

  <chr>      <int>
1 AS          1
2 CT          1
3 DE          1
4 GU          1
5 MD          1
6 NH          1

mapa_de_atrasos <- media_atraso %>%
  left_join(airports, by = c("DESTINATION_AIRPORT" = "IATA_CODE"))

glimpse(mapa_de_atrasos)

Rows: 322
Columns: 8
$ DESTINATION_AIRPORT <chr> "ABE", "ABI", "ABQ", "ABR", "ABY", "ACK",
"ACT", "...
$ soma_total          <dbl> 12874, 9077, 106415, -2227, 7501, 2393,
16392, 122...
$ n_total             <int> 2220, 2224, 18953, 657, 864, 487, 1541, 1267,
3530...
$ media_atraso        <dbl> 5.7990991, 4.0813849, 5.6146784, -3.3896499,
8.681...
$ CITY                <chr> "Allentown", "Abilene", "Albuquerque",
"Aberdeen",...
$ STATE               <chr> "PA", "TX", "NM", "SD", "GA", "MA", "TX",
"CA", "N...
$ LATITUDE            <dbl> 40.65236, 32.41132, 35.04022, 45.44906,
31.53552, ...
$ LONGITUDE           <dbl> -75.44040, -99.68190, -106.60919, -98.42183,
-84.1...

#Criando o mapa
#this <- leaflet(mapa_de_atrasos) %>%
#   addTiles() %>%
#   addMarkers(
#     lng = ~LONGITUDE,
#     lat = ~LATITUDE,
#     popup = ~paste(
#       "Aeroporto:", DESTINATION_AIRPORT, "<br>",
#       "Cidade:", CITY, "<br>",
#       "Estado:", STATE, "<br>",
#       "Atraso médio:", round(media_atraso, 2), "minutos"
#     )
#   )

# Exibir o mapa
#this

```

```
#Horário  
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()  
[1] "2026-09-01 09:51:15 -03"
```